

Absolutely. For your HRMS, I recommend **not giving Lovable one giant prompt**. Use these 10 prompts sequentially. Each step builds on the previous one and keeps the architecture, UI, and business rules consistent.

One important caveat: Lovable is primarily designed around web technologies such as React, so asking it to directly generate a **Blazor Server + MudBlazor 8.8.0 + EF Core + SQL Server** solution may not produce a reliable .NET solution. The prompts below therefore make the target architecture explicit and tell Lovable not to replace your requirements with its preferred stack.

10-Step Lovable HRMS Development Sequence

Step 1 — Architecture, Design System & Application Shell

Paste this first into a new Lovable project:

Edit

You are a Senior Enterprise .NET Solution Architect, HRMS Product Architect, and UX/UI Designer.

We are building a large-scale enterprise Human Resource Management System (HRMS).

The application must be designed around the following target architecture and technology stack:

- C#
- .NET 8
- ASP.NET Core
- Blazor Server
- Razor Components
- MudBlazor 8.8.0
- Entity Framework Core 8
- EF Core Code-First
- Microsoft SQL Server
- Clean Architecture
- Dependency Injection
- FontAwesome
- Bootstrap where appropriate

IMPORTANT:

Do not simplify this into a generic HR application.

Do not remove enterprise functionality.

Do not replace Blazor Server with React, Angular, Vue, Node.js, or another frontend framework.

Do not replace SQL Server with another database.

The objective is to establish the architecture, design system, navigation, layouts, and reusable components that will be used by all subsequent HRMS modules.

=====

1. CLEAN ARCHITECTURE

=====

Design the solution using:

HRMS.sln

HRMS.Domain

HRMS.Application

HRMS.Infrastructure

HRMS.Web

Dependencies:

HRMS.Domain

↓

HRMS.Application

↓

HRMS.Infrastructure

↓

HRMS.Web

Domain must not depend on Infrastructure or UI.

Business logic must not be placed inside Razor components.

Create:

- Login
- Main Layout

- Left navigation drawer
- Top application bar
- User profile menu
- Notification center
- Global search
- Breadcrumbs
- Page header
- Footer
- Mobile navigation
- Light/Dark theme
- Loading indicators
- Error handling
- Snackbar notifications

Create expandable navigation groups:

Dashboard

Time & Attendance

Leave Management

Core HR

Employee Lifecycle

Learning Management

Recruitment

Travel & Expense

Payroll

Reports

System Configuration

Create reusable enterprise components:

- PageHeader
- SearchToolbar
- FilterPanel
- DataGrid
- KPI Card

- Statistics Card
- Employee Card
- Approval Card
- Status Badge
- Confirmation Dialog
- Add/Edit Dialog
- File Upload
- Document Viewer
- Timeline
- Audit Trail
- Empty State
- Loading State
- Error State
- Notification
- Breadcrumb

Use MudBlazor 8.8.0 components wherever possible.

Support:

English

Arabic

English:

LTR

Arabic:

RTL

The entire layout must dynamically switch between LTR and RTL.

The application must work on:

Desktop

Laptop

Tablet

Mobile

Use mobile-first responsive design.

Prepare the application for:

ASP.NET Core Identity

Roles:

System Administrator

HR Administrator

HR Manager

Manager

Supervisor

Employee

Payroll Administrator

Time Office Administrator

Prepare authorization policies and permission-based navigation.

Use:

SQL Server

EF Core Code-First.

Create the foundation for:

ApplicationDbContext

Audit fields:

CreatedBy

CreatedDate

ModifiedBy

ModifiedDate

Use `IEntityTypeConfiguration<TEntity>`.

Follow:

SOLID

DRY

Clean Code

Separation of Concerns

Dependency Injection

Async/Await

Nullable Reference Types

Enterprise exception handling

Logging

Validation

Do not create fake implementations.

Do not create unnecessary placeholder data.

Do not remove requirements.

First establish the architecture and application shell.

Do not build all HRMS modules yet.

Ensure that this foundation can support all subsequent modules without requiring architectural redesign.

Before continuing to another module, verify:

- Navigation
- Layout
- Responsive design
- Localization
- RTL
- Theme
- Authentication foundation
- Authorization foundation
- Component reusability
- Architecture consistency

Step 2 — Core HR / Employee Master

After Step 1 is stable, use:

Edit

Continue developing the existing HRMS application.

Do not redesign the application shell.

Do not change the established architecture or design system.

Implement the CORE HR module, beginning with Employee Master.

Technology requirements remain:

C#

.NET 8

Blazor Server

Razor Components

MudBlazor 8.8.0

EF Core Code-First

SQL Server

Clean Architecture

Create an enterprise Employee Master module.

Employee List:

- Employee Number
- Employee Photo
- Full Name
- Department
- Position
- Employment Status
- Hire Date
- Manager
- Location

Provide:

- Search
- Advanced filtering
- Sorting
- Pagination
- Column selection
- Export
- Add
- Edit
- View
- Deactivate
- Employee status filtering

Create tabs:

About Me

Address / Coordinates

Identity Proofs

Skills & Qualifications

Family

Employment History

Employment Documents

Emergency Contacts

Organization Assignment

Compensation Summary

Attendance Summary

Leave Summary

Create appropriate Domain entities and EF Core configurations.

At minimum consider:

Employee

Department

Position

EmployeeAddress

EmployeeIdentityProof

EmployeeSkill

EmployeeQualification

EmployeeFamily

EmployeeEmploymentHistory

EmployeeDocument

Use proper:

Primary Keys

Foreign Keys

Indexes

Relationships

Required fields

Maximum lengths

Audit fields

Use MudDataGrid.

Use MudDialog for Add/Edit.

Use MudForm for validation.

Use reusable Employee components.

Do not put business logic inside .razor files.

Use .razor.cs code-behind where appropriate.

HR administrators can manage employees.

Managers can view authorized employees.

Employees can view only their own profile.

Implement authorization boundaries.

Do not use fake persistence.

Implement the complete Application → Infrastructure → EF Core → SQL Server flow.

Keep the existing HRMS shell unchanged.

Step 3 — Time & Attendance

This is probably the **most important module in your HRMS**, because it contains many interconnected business rules.

Edit

Continue the existing HRMS application.

Implement the complete TIME & ATTENDANCE module.

Do not modify the established application shell, design system, localization, authorization model, or Clean Architecture.

Employee Self Service

Shift Roster

Face Attendance

Extra Time Request

Time Office

Time Card

Digital ID

Timesheet Calendar

Manual Attendance

Punch In/Out

Reason of Absences

Timesheet Correction

Create:

Employee attendance dashboard

Display:

- Swipe History
- Absence History
- Leave History
- Attendance History
- Leave Balances
- DIL Entitlements
- Shift Pattern
- Dependents

Actions:

- Apply Regularization
- Apply Extra Time
- Apply Work From Home

Create:

Pending Request dashboard

Team View

Implement:

- Shift Pattern
- Shift Timing Template
- Employee Assignment
- Contractor Assignment
- Shift Projection
- Department View
- Employee View
- Date-range filtering

Create:

Face Registration

Face Attendance Report

Configuration

Raw Swipe Data Viewer

Prepare interfaces for future biometric device integration.

Do not hard-code a specific device vendor.

Create:

Extra Time Request

Pending Approval

OT Constraints

OT Configuration

Approval workflow

Create:

Employee Data

Configuration

System Flags

Attendance Processing Rules

Business Rules

Create employee attendance report by period.

Show:

Scheduled Time

Actual Time

Late

Early Out

Absent

No Pay Hours

Overtime

Leave

Holiday

Rest Day

Manage:

Public Holidays

Ramadan Period

Special Working Days

Allow authorized HR users to manually enter:

Date

Time In

Time Out

Reason

Remarks

Maintain audit history.

Create absence reason configuration.

Examples:

Training

Business Trip

Unpaid Sick Leave

Other Authorized Absence

The attendance engine must recognize these entries and prevent incorrect absence marking.

Allow authorized HR users to correct:

Absence
No Pay Hours
Overtime
Allowances

All changes must be audited.

Do not implement attendance calculations inside Razor components.

Create an Application-layer attendance processing service.

Prepare the architecture for future integration with:

Biometric Readers
Mobile Punching
Web Punching
External Attendance Systems

Use asynchronous database operations.

Use SQL Server + EF Core Code-First.

Step 4 — Leave Management

Edit

Continue the existing HRMS.

Implement LEAVE MANAGEMENT.

Preserve all existing architecture and UI conventions.

Create:

Leave Request

Leave Approval

Leave Balance Management

Leave Cancellation

Leave Planner

Employee should be able to:

- Select leave type
- Select dates
- View available balance
- View requested duration
- Enter reason
- Attach documents where applicable
- Submit
- Save Draft
- Cancel pending request

Validate:

- Leave balance
- Overlapping leave
- Holidays
- Weekends
- Employee eligibility
- Minimum/maximum duration
- Required documents
- Approval workflow

Managers and HR should see:

Pending Requests

Request Details

Employee Information

Leave Balance

Leave Calendar

Approval History

Actions:

Approve

Reject

Return

Delegate

HR can manage:

Opening Balance

Accrual

Adjustment

Used Balance

Pending Balance

Available Balance

Carry Forward

Encashment

Allow authorized HR users to cancel approved leave.

Automatically reconcile leave balances.

Maintain audit trail.

Create calendar/dashboard showing planned leave.

Provide:

Employee view

Department view

Manager view

Monthly view

Weekly view

Show overlapping employee leave.

Create appropriate entities:

LeaveType

LeaveRequest

LeaveBalance

LeaveBalanceTransaction

LeaveApproval

LeaveCancellation

Use EF Core Code-First.

Employee

↓

Submit

↓

Supervisor/Manager

↓

HR where required

↓

Approved/Rejected

Use configurable approval rules.

Do not hard-code approval levels.

Step 5 — Employee Lifecycle

Edit

Continue the existing HRMS.

Implement EMPLOYEE LIFECYCLE.

Modules:

Manager Change

Employee Separation

Employee Transfer

Employee Profile

Personal Action Form

Employee Performance Appraisal

Create:

One-to-One Manager Change

Manager Mapping Configuration

Allow authorized HR users to:

- Select employee
- Select new manager
- Define effective date
- Record reason
- Submit change
- View history

Create:

Employee Basic Details

Transaction Details

Attendance View up to 6 months

Leave Details

Separation Workflow

E-Separation Application

Approval Process

Exit Checklist

Audit Trail

Create:

ELC History

Transaction Inquiry

Allow:

Department Transfer

Position Transfer

Manager Transfer

Cost Center Transfer

Location Transfer

Effective Date

Maintain complete history.

Create configurable personnel action transactions.

Examples:

Promotion

Demotion

Salary Change

Department Change

Position Change

Manager Change

Location Change

Employment Status Change

Create:

Performance Cycle

Employee Goals

Manager Evaluation

Employee Self Evaluation

Ratings

Comments

Final Rating

Approval

=====

=====

IMPORTANT

All lifecycle transactions must be effective-dated.

Maintain complete historical records.

Do not overwrite historical employee information.

Step 6 — Payroll & Compensation

Payroll deserves its own phase because it should **not be mixed into the Employee Master implementation.**

Edit

Continue the existing HRMS.

Implement PAYROLL AND COMPENSATION.

Modules:

Employee Compensation Setup

Payroll Processing

Payslip Generation Inquiry

Global Bulk Payslip

Payroll Configuration

Timesheet Upload

Create compensation information for employees.

Support:

Basic Salary

Allowances

Deductions

Benefits

Contributions

Effective Dating

Create payroll processing workflow:

Payroll Period

Employee Selection

Timesheet Input

Attendance Input

Leave Input

Overtime Input

Allowances

Deductions

Adjustments

Calculation

Validation

Approval

Finalization

Payslip Generation

Create:

Bulk Payslips

Processed Saved Payslip

Payslip Inquiry

Global Bulk Payslip

Support:

VPF Contribution

Mid Payout Payslip

Bonus Payslip

Loan Statement

PF Statement

AL Statement

CTC Breakup

Tax Computation

Payslips

Payroll information is highly restricted.

Employees can only view their own payslips.

Managers cannot view employee payroll details unless explicitly authorized.

Payroll administrators have appropriate access.

Do not hard-code payroll formulas.

Create configurable payroll rules.

Separate payroll calculation logic from UI.

Create an Application-layer payroll calculation service.

Maintain audit history for payroll changes.

Step 7 — Recruitment, Learning & Employee Engagement

Edit

Continue the existing HRMS.

Implement:

RECRUITMENT

LEARNING MANAGEMENT

EMPLOYEE ENGAGEMENT

Training Registration

Training Providers

Training Courses

Employee Training Registration

Training Attendance

Training History

Training Reports

Track:

Course

Provider

Date

Duration

Cost

Employee

Status

Attendance

Certification

Create:

Document Section

Employee Master

Geo Configuration

Integrations

Manager Change

Proxy Mapping

System Monitor

User Status

Employee Separation

Probationary Assessment

Employee Contract Renewal

Create:

Probation Period

Assessment Criteria

Manager Assessment

HR Review

Recommendation

Confirmation

Extension

Termination

Show employees with upcoming contract expiry.

Provide:

Expiry Dashboard

Renewal Request

Approval

New Contract

History

Notifications

Ensure employees can only access their own permitted information.

HR and authorized managers can access appropriate organizational information.

Step 8 — Travel & Expense

Edit

Continue the existing HRMS.

Implement TRAVEL & EXPENSE.

Modules:

Travel Request

My Travel History

Bulk Travel Data Mapping

Parallel Approver Mapping

Travel Configuration

Create a multi-step request wizard.

Step 1:

Travel Details

Step 2:

Destination

Step 3:

Dates

Step 4:

Purpose

Step 5:

Estimated Expenses

Step 6:

Supporting Documents

Step 7:

Approval

Employee can view:

Draft

Submitted

Approved

Rejected

Completed

Cancelled

Support:

Manager Approval

Parallel Approval

HR Approval

Finance Approval

Configurable approval levels.

Configure:

Travel Types

Expense Types

Countries

Cities

Currency

Limits

Approval Rules

Create bulk travel data mapping.

Provide:

Upload

Validation

Error Report

Preview

Import

Audit

=====

=====

IMPORTANT

Use reusable workflow components that can later be shared with Leave, Overtime, Separation and other HR workflows.

Step 9 — System Configuration, Security & Administration

Edit

Continue the existing HRMS.

Implement SYSTEM CONFIGURATION AND ADMINISTRATION.

Configure:

Company

Business Unit

Department

Cost Center

Location

Position

Job

Grade

Manager

Superintendent

Configure:

Portal Name

Logo

Favicon

Language

Default Language

Date Format

Time Format

Currency

Timezone

Email Settings

Notification Settings

Configure:

Employee Number Rules

Employee Status

Employee Types

Nationality

Gender

Marital Status

Employment Types

Configure:

Salary Components

Allowances

Deductions

Benefits

Contributions

Configure:

Attendance Rules

Shift Rules

OT Constraints

Late Rules

Early Out Rules

Absence Rules

Grace Periods

Work From Home Rules

Configure:

Travel Rules

Expense Limits

Approval Rules

Configure:

Payroll Rules

Payroll Components

Tax Rules

Contribution Rules

Configure:

Performance Cycles

Rating Scales

Goals

Competencies

Configure employee onboarding fields and workflows.

Configure:

Transfers

Promotions

Manager Changes

Separations

Contract Renewal

Allow administrators to configure:

Logo

Primary Theme

Secondary Theme

Light/Dark Mode

Arabic/English

RTL/LTR

Create:

Users

Roles

Permissions

Role Permissions

User Roles

Menu Permissions

Page Permissions

Action Permissions

Audit Logs

Login History

System Logs

=====

=====

IMPORTANT

Use permission-based authorization.

Do not rely only on hiding menu items.

Backend authorization must also be enforced.

Step 10 — Enterprise Integration, Testing & Production Hardening

This is the final step and is particularly important for your HRMS because it will eventually connect to other systems.

Edit

Continue the existing HRMS.

This is the ENTERPRISE HARDENING AND INTEGRATION phase.

Do not redesign existing modules.

Review the complete application for consistency, security, performance, maintainability and scalability.

=====

1. INTEGRATIONS

=====

Prepare interfaces for:

Biometric Attendance Devices

Face Readers

Email

SMS

Mobile Applications

External Payroll

ERP

Active Directory / Microsoft Entra ID

Document Storage

Notification Services

Where external integration is required, create clean service abstractions.

Example:

IAttendanceDeviceService

IEmailService

INotificationService

IDocumentStorageService

IPayrollIntegrationService

Do not place external API calls inside Razor components.

Create centralized audit logging.

Record:

User

Action

Module

Entity

Entity ID

Old Value

New Value

Timestamp

IP Address where appropriate

Implement centralized exception handling.

Provide user-friendly error messages.

Never expose:

SQL exceptions

connection strings

stack traces

internal implementation details

Review all database queries.

Use:

AsNoTracking()

where appropriate.

Use:

Pagination

Projection

Server-side filtering

Server-side sorting

Async queries

Avoid N+1 queries.

Do not load entire employee populations into browser memory.

Review:

Authentication

Authorization

Role permissions

Page permissions

Action permissions

Data access boundaries

Employee self-service security

Payroll security

Document security

File upload security

Review every major form.

Ensure:

Required fields

Business validation

Duplicate validation

Date validation

Authorization validation

Workflow validation

Test:

Desktop

Tablet

Mobile

Arabic RTL

English LTR

Light Theme

Dark Theme

Review:

Keyboard navigation

ARIA labels

Focus management

Color contrast

Screen-reader compatibility

Form validation messages

=====

=====

10. TESTING

Create a testing strategy covering:

Unit Tests

Application Service Tests

Domain Tests

Integration Tests

Database Tests

Authorization Tests

UI Tests

Workflow Tests

=====

=====

11. DATABASE

Review:

Indexes

Foreign Keys

Relationships

Concurrency

Transactions

Audit fields

Soft deletion

Migration strategy

=====

=====

12. PRODUCTION READINESS

Prepare for:

Logging

Monitoring

Health Checks

Configuration Management

Environment Variables

Development

Testing

Production

Backup/Restore

Deployment

=====

13. FINAL ARCHITECTURE REVIEW

Verify that the application follows:

Clean Architecture

SOLID

DRY

Separation of Concerns

Dependency Injection

Secure Coding

Async Programming

Enterprise Coding Standards

=====

FINAL REQUIREMENT

Do not simply report problems.

For each identified architectural or implementation problem:

1. Explain the problem.
2. Explain why it matters.
3. Correct it.
4. Verify that the correction does not break existing functionality.

The final HRMS should be structured as an enterprise-grade, maintainable and scalable application.

Recommended order for your actual development

I would use the sequence like this:

Step	Module	Priority
1	Architecture + Shell + Design System	 Critical
2	Core HR / Employee Master	 Critical
3	Time & Attendance	 Critical
4	Leave Management	 Critical
5	Employee Lifecycle	 High
6	Payroll & Compensation	 Critical
7	Learning + Recruitment + Engagement	 Medium
8	Travel & Expense	 Medium
9	System Configuration	 Critical
10	Integration + Security + Testing	 Critical

One change I'd make for your particular HRMS

Because you already have substantial experience with **Blazor Server**, **MudBlazor**, **EF Core Code-First** and **SQL Server**, I would actually use Lovable primarily for **UI/UX prototyping**, then use your existing Visual Studio solution as the authoritative production codebase.

For example:

Lovable

Employee Master → beautiful UI/workflow

↓

You + ChatGPT

MudBlazor 8.8.0 components

↓

Blazor .razor + .razor.cs

↓

Application Services

↓

Domain

↓

EF Core

↓

SQL Server

That gives you the benefit of AI-generated UI without allowing an AI application builder to accidentally undermine the **Clean Architecture and HR business rules** that you've already built into your HRMS.

Also, for the next stage, I would recommend creating a "**Master HRMS Prompt**" containing your complete module hierarchy, naming conventions, database conventions, MudBlazor conventions, approval workflow conventions, security model, and coding standards. Then you can prepend/reference that standard whenever you ask AI to build a new HRMS screen. This will significantly reduce inconsistencies across hundreds of screens.

